Embedded Systems 14th Week

– Interrupts and timers –

Jaeseok Yun

Soonchunhyang University

Agenda

- Interrupts and ISR (interrupt service routines)
- External interrupts
- Pin change interrupts (PICINTs)
- Timer overflow interrupts (TOIs)
- TimerOne library
- `interrupts()` and `noInterrupts()`

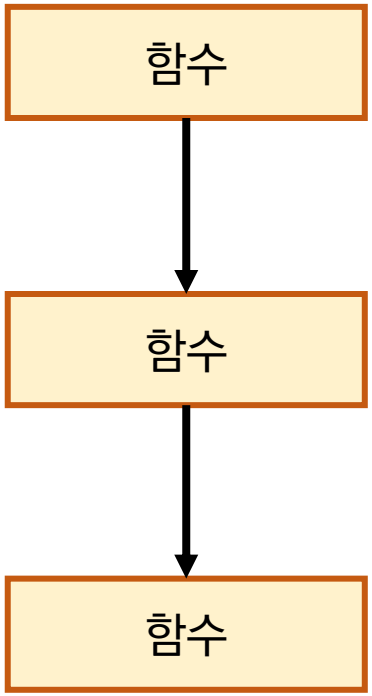
인터럽트

Interrupt

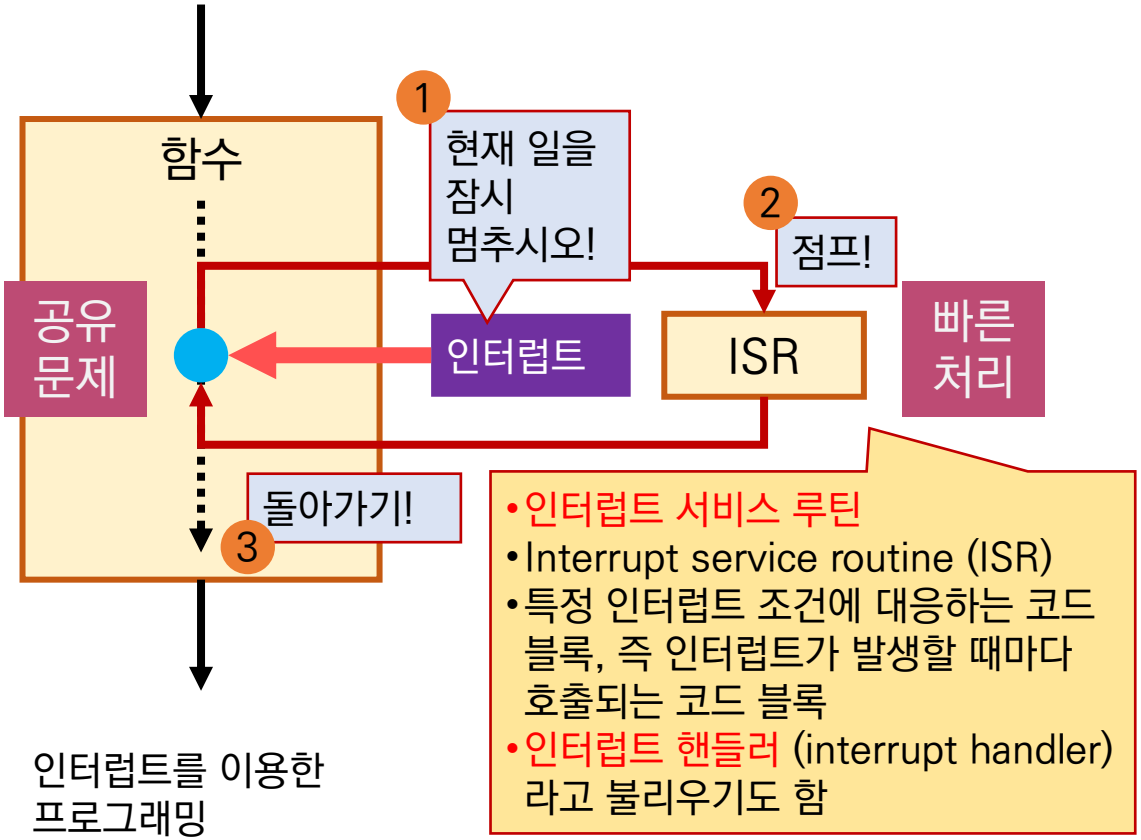
인터럽트 Interrupt

■ 마이크로컨트롤러에서 인터럽트란?

- CPU가 현재 하는 일을 '일시적으로' 멈추게 하는 신호



절차적 프로그래밍



인터럽트를 이용한 프로그래밍

외부 인터럽트

External interrupts

외부 인터럽트와 스케치

외부 인터럽트 번호[1]	INT.0	INT.1	INT.2	INT.3	INT.4	INT.5
우노 (핀 번호)	2	3				
레오나르도 (핀 번호)	3	2	0	1	7	
메가 (핀 번호)	2	3	21	20	19	18

■ 스케치 코드

```
void setup() {
  // put your setup code here, to run once:

}
```

한번 만!

```
void loop () {
  // put your main code here, to run repeatedly:

}
```

반복!

```
void isr () {

}
```

인터럽트가 발생할 때마다 호출

1 현재 일을 잠시 멈추시오!

인터럽트

2 점프!

3 돌아가기!

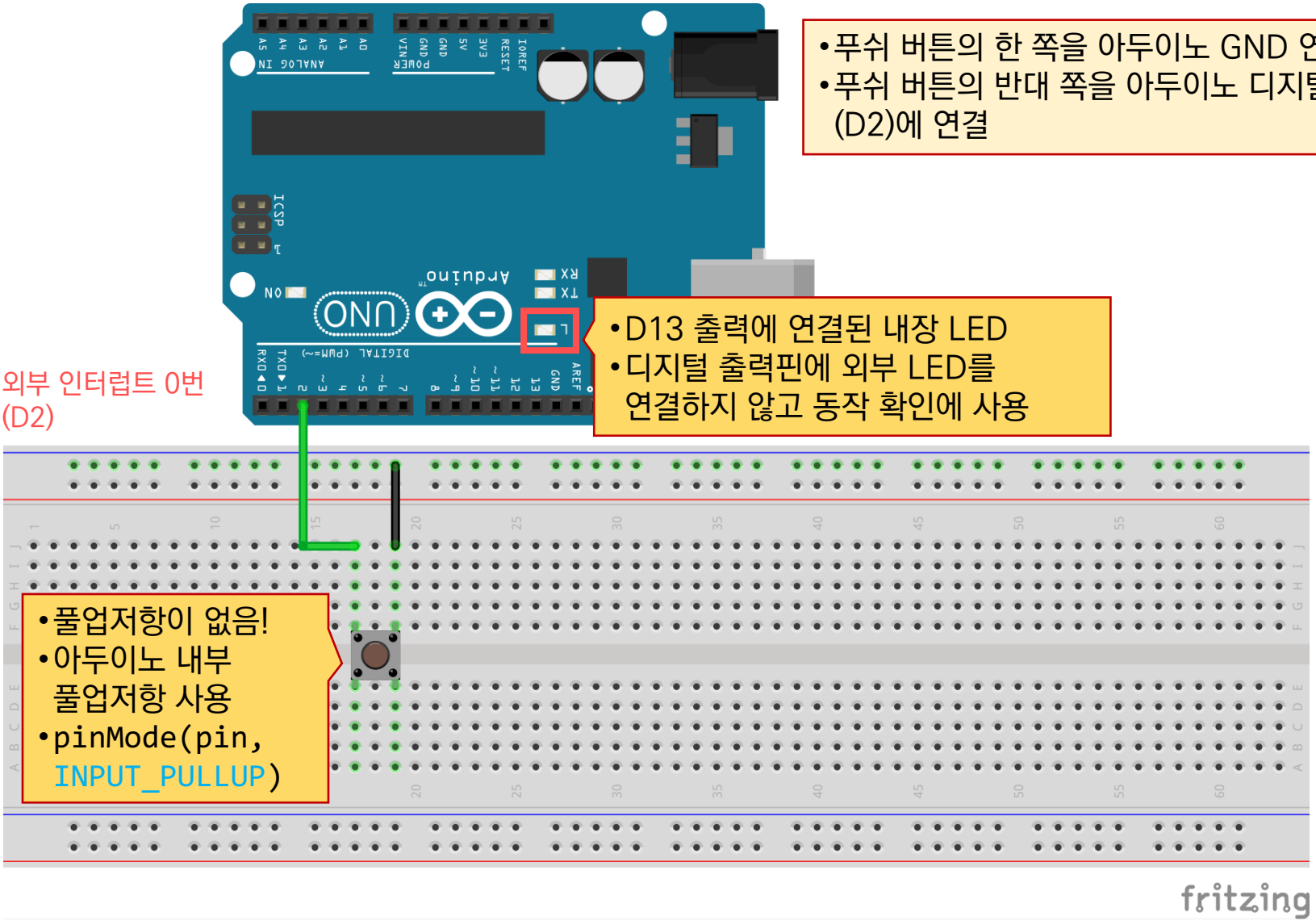
인터럽트 서비스 루틴
Interrupt service routine

- Q: 언제 인터럽트가 발생할까?
- A: 인터럽트는 미리 정의됨, 예로서 특정 핀의 상태가 '변화할 때 (1→0)' 인터럽트가 발생되며 대응 isr()로 점프

[1] https://www.arduino.cc/reference/en/language/functions/external-interrupts/attachinterrupt/

외부 인터럽트를 위한 아두이노 연결

Practice



- 푸쉬 버튼의 한 쪽을 아두이노 GND 연결
- 푸쉬 버튼의 반대 쪽을 아두이노 디지털 입력 핀 (D2)에 연결

- D13 출력에 연결된 내장 LED
- 디지털 출력핀에 외부 LED를 연결하지 않고 동작 확인에 사용

외부 인터럽트 스케치 예

```
const byte pinLED = 13;
const byte pinINT = 2;
volatile byte state = LOW;

void setup() {
  pinMode(pinLED, OUTPUT);

  pinMode(pinINT, INPUT_PULLUP);
  attachInterrupt(1, countVib, FALLING);
  attachInterrupt(digitalPinToInterrupt(pinINT), blink, CHANGE);
}

void loop() {
  digitalWrite(pinLED, state);
}

void blink() { state = !state; }
```

비교

- **volatile**[1] 키워드는 변수를 선언할 때 사용되는 한정어 (qualifier)
- 일반 변수 선언 시 컴파일러는 변수가 저장/가공되는 임시 메모리 영역인 '저장 레지스터 (storage register)'를 사용하게 기계어 번역
- **volatile** 변수는 컴파일러가 기계어로 번역 할 때 저장 레지스터가 아닌 RAM에서 해당 변수를 로드하도록 특별히 지시하는 효과
- 저장 레지스터에 저장된 변수는 특정 조건에서 부정확할 수 있음
- 예로서 동시 수행 스레드에서 공통 필요 변수는 반드시 **volatile** 키워드로 선언되야 하며 아두이노는 인터럽트가 유일한 경우임

- **INPUT_PULLUP**[2] 인수는 pinMode()의 첫 번째 인수인 디지털 핀에 '풀업 저항 (약 20kΩ)'을 소프트웨어로 활성화 하는 효과
- 핀 동작은 '1'은 연결된 센서가 꺼짐, '0'은 켜짐으로 해석
- 입력 또는 통신 데이터가 없으면 '1' 유지 (풀다운 저항 없음)

- **attachInterrupt()**[3]은 '외부 인터럽트'와 '해당 인터럽트가 발생할 때 대응하는 ISR'을 연결 수행
- attachInterrupt(digitalPinToInterrupt(pin), ISR, mode); /* 추천 */
- attachInterrupt(interrupt, ISR, mode); /* interrupt는 인터럽트 번호, 비추천 */
- attachInterrupt(pin, ISR, mode); /* pin은 핀 번호, Due, Zero, 101, MKR1000 만 가능 */
- **ISR**은 첫 번째 인수 '외부 인터럽트 번호'에 대응하는 ISR (여기서는 blink)의 이름 (즉, 포인터)
- **mode**는 인터럽트가 언제 '트리거' 되야 하는지 정의 (**LOW**, **CHANGE**, **RISING**, **FALLING**, **HIGH**)

- **blink()** 함수는 아두이노의 '0번' 외부 인터럽트 (핀 번호로는 우노의 D2)에 대응하는 ISR로 등록됨
- 입력 신호가 '변화 할 때 (1→0 또는 0→1)'에 인터럽트 발생
- **state**가 loop()와 ISR인 blink() 모두 접근하기 위해서는 반드시 '**volatile**' 한정자를 사용해서 선언해야 함을 주의!

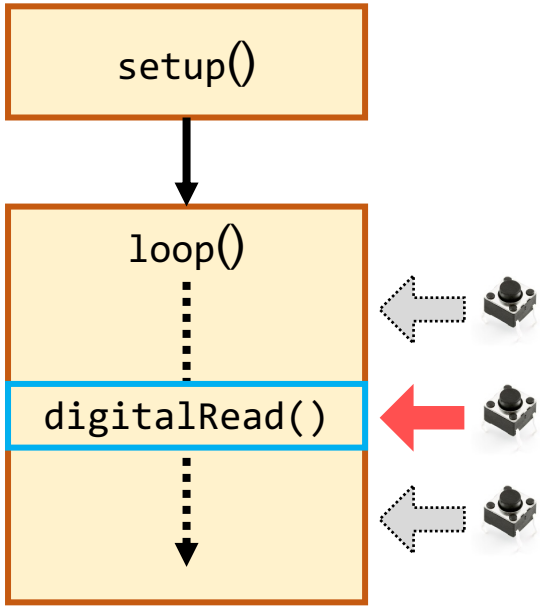
결과: 버튼을 누르면 LED 점등

[1] https://www.arduino.cc/reference/en/language/variables/variable-scope--qualifiers/volatile/
[2] https://www.arduino.cc/en/Tutorial/DigitalPins
[3] https://www.arduino.cc/reference/en/language/functions/external-interrupts/attachinterrupt/

인터럽트 vs. 디지털 입력

■ 인터럽트의 효과

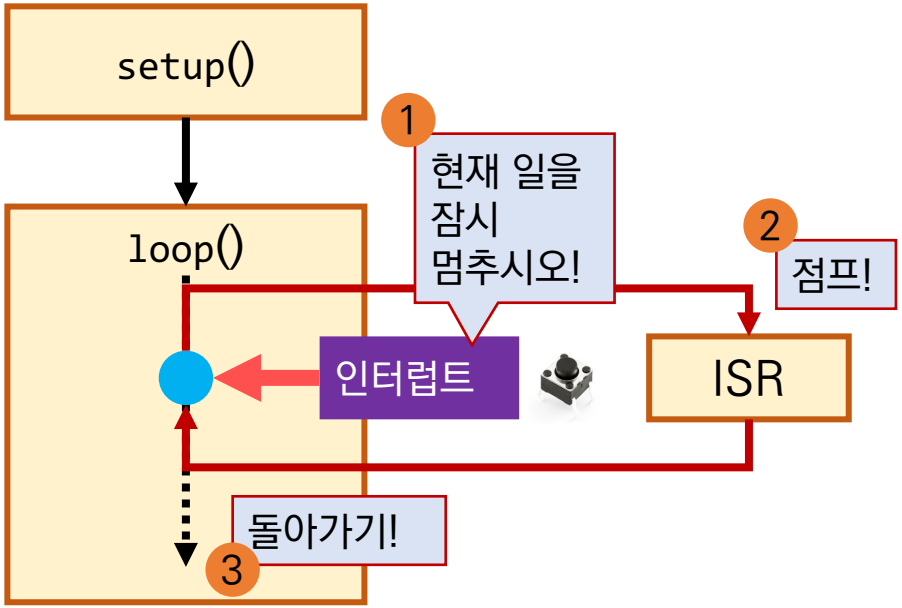
digitalRead()



절차적 프로그래밍

버튼의 값을 읽어 들이는
'정확한 시간'에 눌러야 함

ISR()



인터럽트를 이용한 프로그래밍

버튼을 누를 때마다 외부 인터럽트가
발생하여 '즉시' 대응

핀 변화 인터럽트

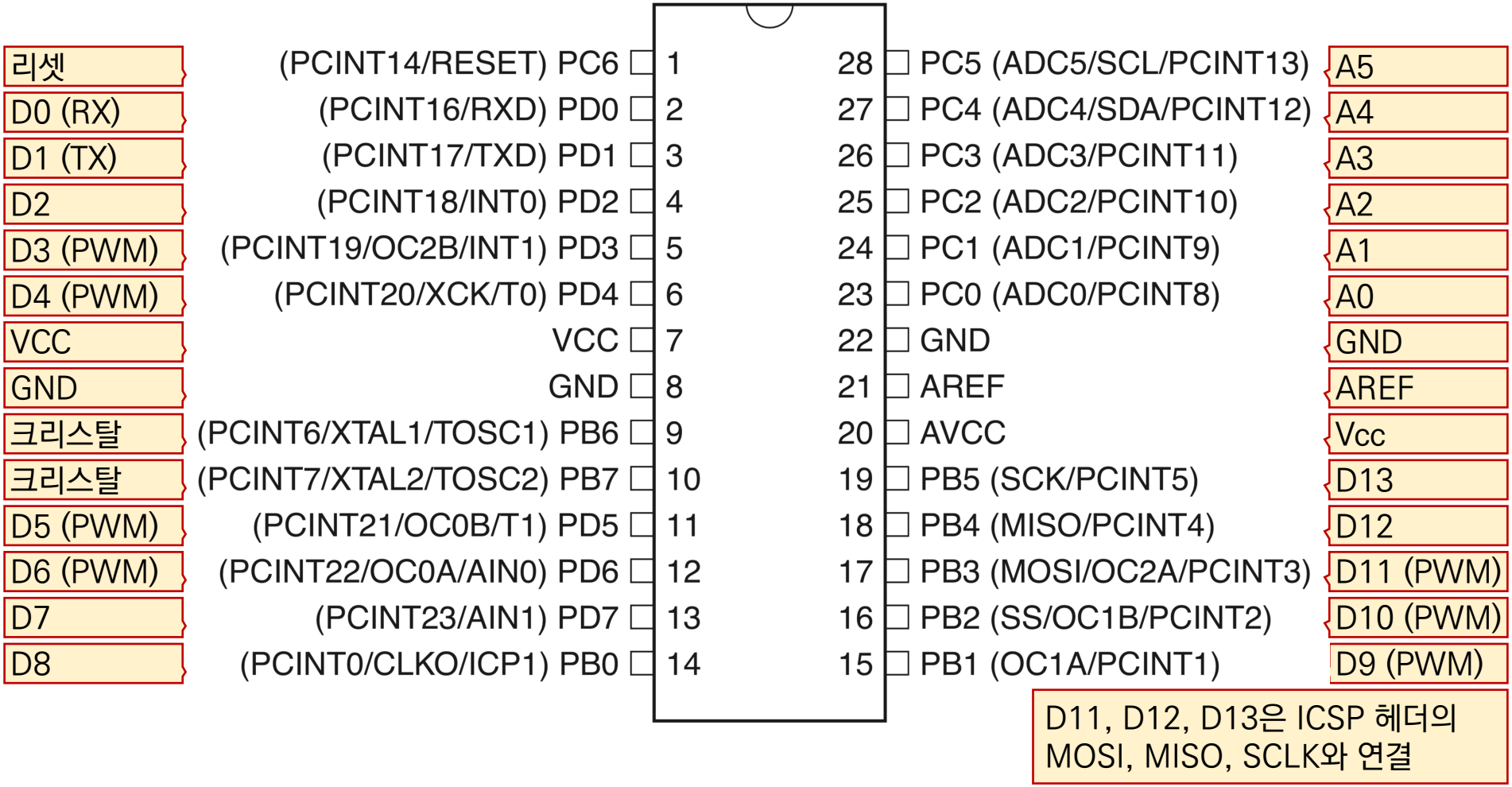
Pin Change Interrupts (PCINTs)

부족한 외부 인터럽트 (?)

- 아두이노 플랫폼의 한계
  - 아두이노 우노의 경우 2개의 인터럽트 (INT.0, INT.1) 만 제공
- Q: 더 이상 인터럽트를 사용할 수 없을까?
- A: 핀 변화 인터럽트 (Pin Change Interrupts)
  - 디지털 또는 아날로그 핀의 값이 변화할 때 발생하는 인터럽트
  - AVR 시리즈 마이크로컨트롤러에 구현되어 있음
  - 아두이노 플랫폼 (스케치)는 지원하지 않으며 직접 AVR 프로그래밍이 필요

외부 인터럽트 번호[1]	INT.0	INT.1	INT.2	INT.3	INT.4	INT.5
우노 (핀 번호)	2	3				
레오나르도 (핀 번호)	3	2	0	1	7	
메가 (핀 번호)	2	3	21	20	19	18

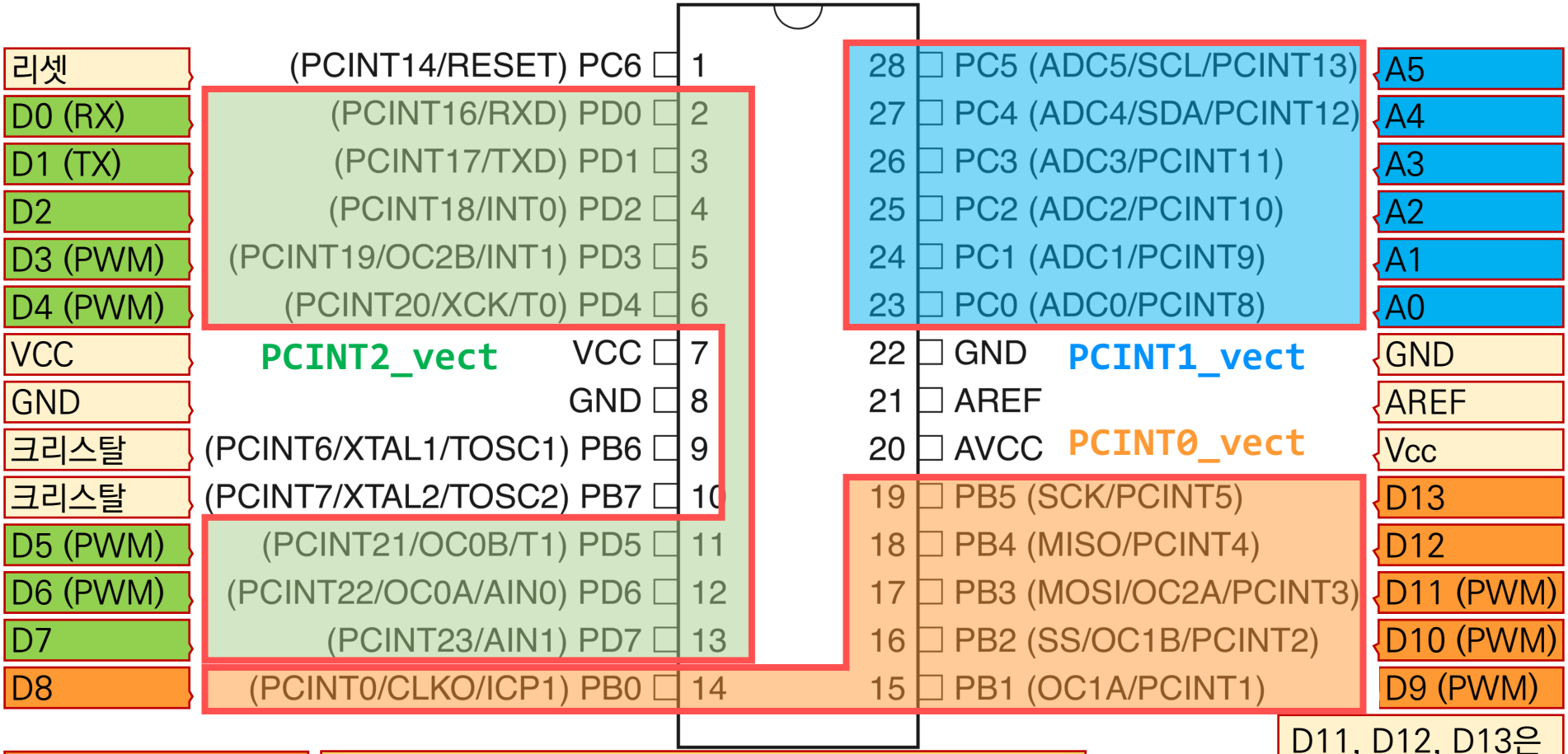
ATmega328P 핀 배열[1]



D11, D12, D13은 ICSP 헤더의 MOSI, MISO, SCLK와 연결

[1] https://www.microchip.com/wwwproducts/en/atmega328p
[2] https://www.teachmemicro.com/arduino-interrupt-tutorial/

핀 변화 인터럽트 PCINTs (Pin Change INTerrupts)



ISR(PCINT0_vect)
ISR(PCINT1_vect)
ISR(PCINT2_vect)

ATmega328p 각 핀에 PCINTx가 부여되어 있으나 실제로는 3개의 인터럽트 벡터가 정의되어 있고, 따라서 3개의 ISR이 인터럽트에 발생에 대응

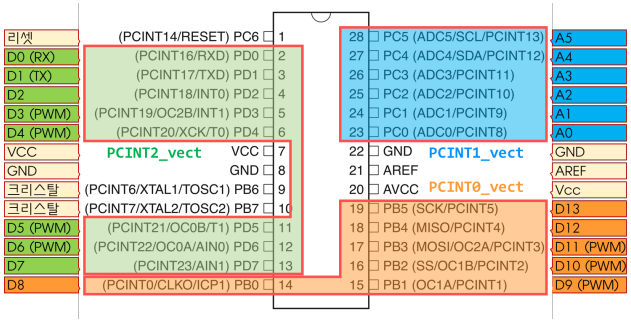
D11, D12, D13은 ICSP 헤더의 MOSI, MISO, SCLK와 연결

[1] https://www.microchip.com/wwwproducts/en/atmega328p
[2] https://www.teachmemicro.com/arduino-interrupt-tutorial/

핀 변화 인터럽트 레지스터 PCICR

■ PCICR (Pin Change Interrupt Control Register)

- 핀 변화 인터럽트 발생 여부를 제어하는 레지스터
- PCIE0 (bit 0)
  - ✓ PCINT0-7까지 핀 변화가 있을 때 인터럽트 발생
  - ✓ (크리스탈 핀을 제외하고) D8-D13 핀에 해당
- PCIE1 (bit 1)
  - ✓ PCINT8-14까지 핀 변화가 있을 때 인터럽트 발생
  - ✓ (리셋 핀을 제외하고) A0-A5 핀에 해당
- PCIE2 (bit 2)
  - ✓ PCINT16-23까지 핀 변화가 있을 때 인터럽트 발생
  - ✓ D0-D7 핀에 해당



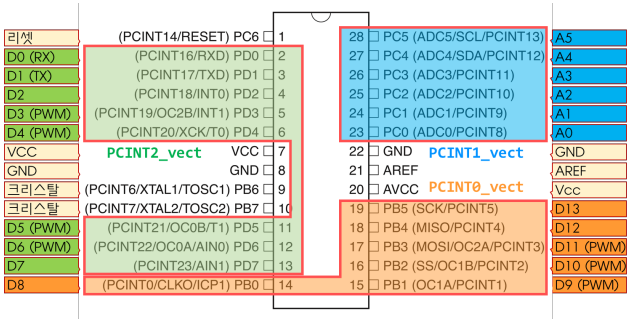
7	6	5	4	3	2	1	0
					PCIE2	PCIE1	PCIE0

핀 변화 인터럽트 제어 레지스터
Pin Change Interrupt Control Register (PCICR)

핀 변화 인터럽트 레지스터 PCIFR

■ PCIFR (Pin Change Interrupt Flag Register)

- 핀 변화 인터럽트가 발생 했을 때 표시 레지스터
- PCIF0 (bit 0)
  - ✓ PCINT0-7까지 PCI가 발생했을 때 1로 설정
  - ✓ (크리스탈 핀을 제외하고) D8-D13 핀에 해당
- PCIF1 (bit 1)
  - ✓ PCINT8-14까지 PCI가 발생했을 때 1로 설정
  - ✓ (리셋 핀을 제외하고) A0-A5 핀에 해당
- PCIF2 (bit 2)
  - ✓ PCINT16-23까지 PCI가 발생했을 때 1로 설정
  - ✓ D0-D7 핀에 해당



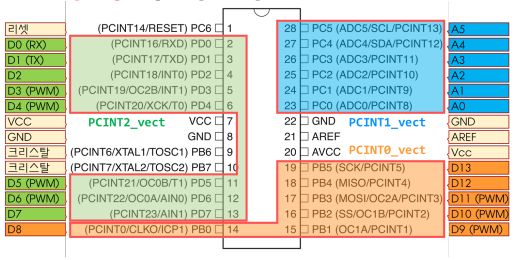
7	6	5	4	3	2	1	0
					PCIF2	PCIF1	PCIF0

핀 변화 인터럽트 플래그 레지스터
Pin Change Interrupt Flag Register (PCIFR)

핀 변화 인터럽트 레지스터 PCMSK

■ PCMSKs (Pin Change Interrupt Mask Registers)

- PCI 세 그룹 내에서 특정 핀에 대해서만 인터럽트 발생 여부를 제어하기 위한 레지스터



- PCMSK0
  - ✓ PCINT0-7 중 PCI 발생 여부 제어 (마스킹)
  - ✓ (크리스탈 핀을 제외하고) D8-D13 핀에 해당

7	6	5	4	3	2	1	0
PCINT7	PCINT6	PCINT5	PCINT4	PCINT3	PCINT2	PCINT1	PCINT0

- PCMSK1
  - ✓ PCINT8-14 중 PCI 발생 여부 제어 (마스킹)
  - ✓ (리셋 핀을 제외하고) A0-A5 핀에 해당

7	6	5	4	3	2	1	0
	PCINT14	PCINT13	PCINT12	PCINT11	PCINT10	PCINT9	PCINT8

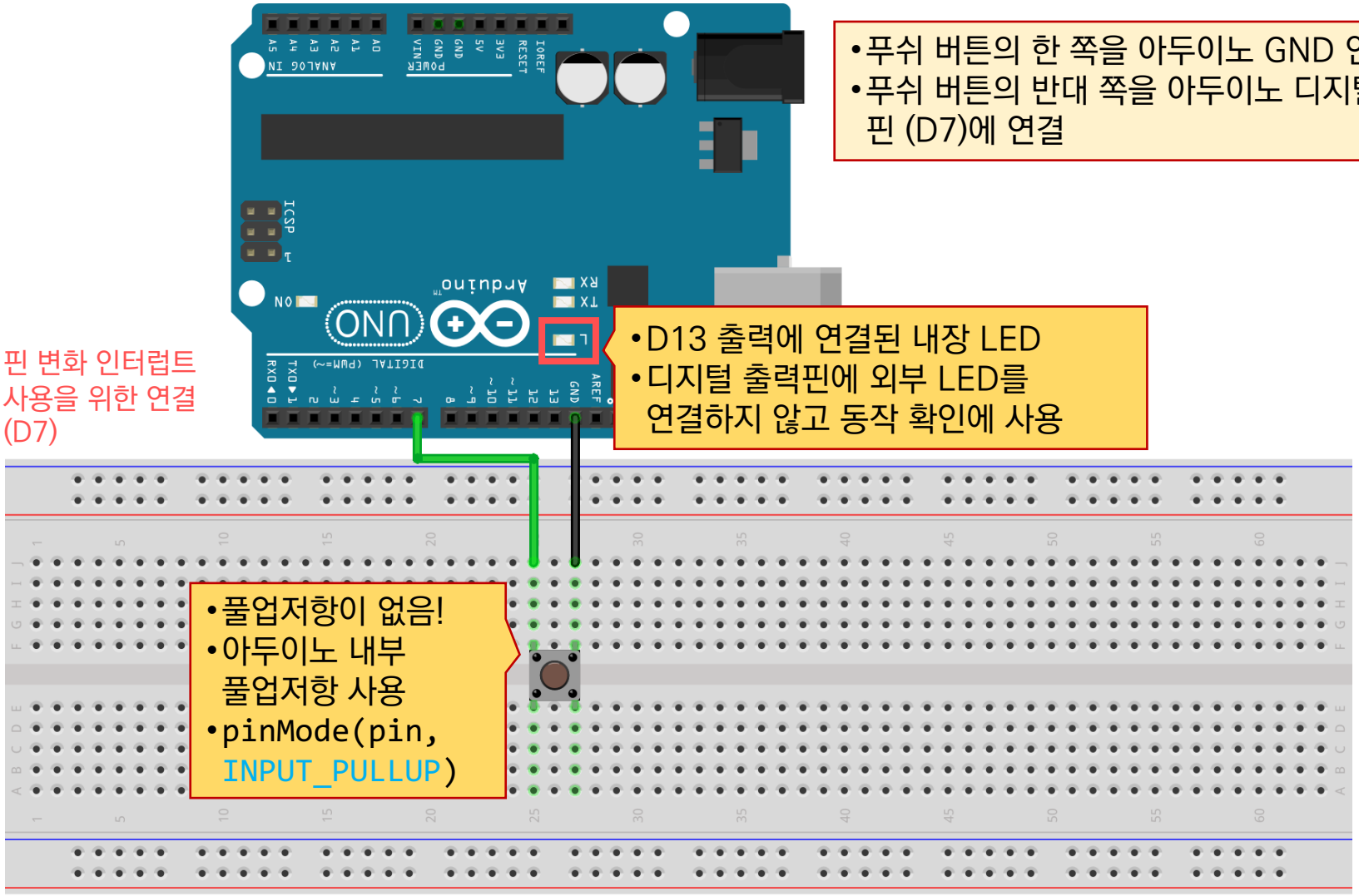
- PCMSK2
  - ✓ PCINT16-23 중 PCI 발생 여부 제어 (마스킹)
  - ✓ D0-D7 핀에 해당

7	6	5	4	3	2	1	0
PCINT23	PCINT22	PCINT21	PCINT20	PCINT19	PCINT18	PCINT17	PCINT16

PCI를 위한 아두이노 연결 A0 PCI 처리

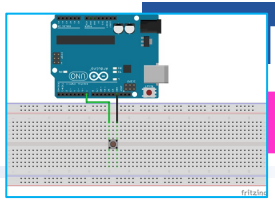
Practice

- 푸쉬 버튼의 한 쪽을 아두이노 GND 연결
- 푸쉬 버튼의 반대 쪽을 아두이노 디지털 입출력 핀 (D7)에 연결



PCI 처리 스케치 예제 blink

버튼을 누르면 LED 점등 결과

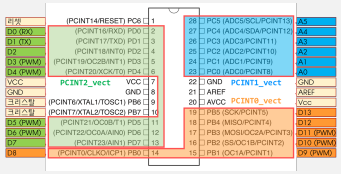


```
const byte pinLED = 13;
const byte pinINT = 7;
volatile byte state = LOW;

void setup() {
  PCICR |= _BV(2);
```

- PCI 인터럽트를 발생시킬 핀을 디지털 입력 D7로 정의
- PCI 인터럽트의 ISR에서 접근할 수 있는 volatile 변수 선언

- #define _BV(bit) (1 << (bit)) 를 활용해 레지스터 설정
- PCI 인터럽트 제어 레지스터의 경우, D7은 PCIE2에 해당하므로 PCICR의 2-th 비트 (PCIE2)를 '1' 설정



7	6	5	4	3	2	1	0
					PCIE2	PCIE1	PCIE0

```
  PCMSK2 |= _BV(7);
```

- PCI 인터럽트 마스크 레지스터의 경우, D7을 위한 마스크는 PCMSK2이고, 7-th 비트 (PCINT23)를 '1' 설정

7	6	5	4	3	2	1	0
PCINT23	PCINT22	PCINT21	PCINT20	PCINT19	PCINT18	PCINT17	PCINT16

```
  pinMode(pinINT, INPUT_PULLUP);
  pinMode(pinLED, OUTPUT);
}

void loop() {
  digitalWrite(pinLED, state);
}

ISR(PCINT2_vect) {
  state = !state;
}
```

- ISR() 는 아두이노 인터럽트 벡터에 대응하는 ISR 함수
- 인수로는 인터럽트 벡터 값이 올 수 있으며 우노의 경우 ATmega328p 마이크로프로세서를 위해 〈iom328p.h〉 헤더파일에 #define 문으로 정의되어 있음
- 이 예제에서는 D7핀이 속한 PCI 인터럽트 그룹이 PCINT2 이므로 'PCINT2_vect'를 인수로 전달
- 뒤에 나오는 iom328p.h 헤더파일 내용 확인!

[1] https://www.teachmemicro.com/arduino-interrupt-tutorial/

Interrupt Vectors 인터럽트 벡터 (iom328p.h[1])

```
/* Interrupt Vectors */
/* Interrupt Vector 0 is the reset vector. */

#define INT0_vect_num     1
#define INT0_vect         _VECTOR(1)   /* External Interrupt Request 0 */

#define INT1_vect_num     2
#define INT1_vect         _VECTOR(2)   /* External Interrupt Request 1 */

#define PCINT0_vect_num   3
#define PCINT0_vect       _VECTOR(3)   /* Pin Change Interrupt Request 0 */

#define PCINT1_vect_num   4
#define PCINT1_vect       _VECTOR(4)   /* Pin Change Interrupt Request 1 */

#define PCINT2_vect_num   5
#define PCINT2_vect       _VECTOR(5)   /* Pin Change Interrupt Request 2 */

#define WDT_vect_num      6
#define WDT_vect          _VECTOR(6)   /* Watchdog Time-out Interrupt */
```

외부 인터럽트 요청 0

외부 인터럽트 요청 1

핀 변화 인터럽트 요청 0

핀 변화 인터럽트 요청 1

핀 변화 인터럽트 요청 2

와치독 타임아웃 인터럽트

[1] C:\Program Files (x86)\Arduino\hardware\tools\avr\avr\include\avr

Interrupt Vectors 인터럽트 벡터 (iom328p.h[1])

```
#define TIMER2_COMPA_vect_num 7
#define TIMER2_COMPA_vect _VECTOR(7)   /* Timer/Counter2 Compare Match A */

#define TIMER2_COMPB_vect_num 8
#define TIMER2_COMPB_vect _VECTOR(8)   /* Timer/Counter2 Compare Match A */

#define TIMER2_OVF_vect_num   9
#define TIMER2_OVF_vect   _VECTOR(9)   /* Timer/Counter2 Overflow */

#define TIMER1_CAPT_vect_num  10
#define TIMER1_CAPT_vect  _VECTOR(10)  /* Timer/Counter1 Capture Event */

#define TIMER1_COMPA_vect_num 11
#define TIMER1_COMPA_vect _VECTOR(11)  /* Timer/Counter1 Compare Match A */

#define TIMER1_COMPB_vect_num 12
#define TIMER1_COMPB_vect _VECTOR(12)  /* Timer/Counter1 Compare Match B */

#define TIMER1_OVF_vect_num   13
#define TIMER1_OVF_vect   _VECTOR(13)  /* Timer/Counter1 Overflow */

#define TIMER0_COMPA_vect_num 14
#define TIMER0_COMPA_vect _VECTOR(14)  /* TimerCounter0 Compare Match A */

#define TIMER0_COMPB_vect_num 15
#define TIMER0_COMPB_vect _VECTOR(15)  /* TimerCounter0 Compare Match B */

#define TIMER0_OVF_vect_num  16
#define TIMER0_OVF_vect   _VECTOR(16)  /* Timer/Couner0 Overflow */
```

- 타이머2 비교 인터럽트 A
- 타이머2 비교 인터럽트 B
- 타이머2 오버플로우
- 타이머1 캡처 이벤트
- 타이머1 비교 인터럽트 A
- 타이머1 비교 인터럽트 B
- 타이머1 오버플로우
- 타이머0 비교 인터럽트 A
- 타이머0 비교 인터럽트 B
- 타이머0 오버플로우

[1] C:\Program Files (x86)\Arduino\hardware\tools\avr\avr\include\avr

Interrupt Vectors 인터럽트 벡터 (iom328p.h[1])

```
#define SPI_STC_vect_num  17
#define SPI_STC_vect      _VECTOR(17)  /* SPI Serial Transfer Complete */

#define USART_RX_vect_num 18
#define USART_RX_vect     _VECTOR(18)  /* USART Rx Complete */

#define USART_UDRE_vect_num   19
#define USART_UDRE_vect   _VECTOR(19)  /* USART, Data Register Empty */

#define USART_TX_vect_num 20
#define USART_TX_vect     _VECTOR(20)  /* USART Tx Complete */

#define ADC_vect_num      21
#define ADC_vect          _VECTOR(21)  /* ADC Conversion Complete */

#define EE_READY_vect_num 22
#define EE_READY_vect     _VECTOR(22)  /* EEPROM Ready */

#define ANALOG_COMP_vect_num  23
#define ANALOG_COMP_vect  _VECTOR(23)  /* Analog Comparator */

#define TWI_vect_num      24
#define TWI_vect          _VECTOR(24)  /* Two-wire Serial Interface */

#define SPM_READY_vect_num    25
#define SPM_READY_vect    _VECTOR(25)  /* Store Program Memory Read */

#define _VECTORS_SIZE (26 * 4)
```

- SPI 전송 완료
- UART 수신 완료
- UART 데이터 레지스터
- UART 전송 완료
- ADC 변환 완료
- EEPROM 준비
- 아날로그 비교기
- I²C 인터페이스
- SPM 대기

[1] C:\Program Files (x86)\Arduino\hardware\tools\avr\avr\include\avr

핀 변화 인터럽트 라이브러리[1,2]

HOME STORE SOFTWARE EDUCATION RESOURCES COMMUNITY HELP

PinChangeInt Library

What's New? Nov. 16, 2012: Version 2.17beta has been released.This release incorporates a number of recent changes, including: 2.15beta: The attachInterrupt() function now allows the user to change user function pointers. 2.13 beta: Major bugfix over previous versions. Look at the Release Notes for the download or read the RELEASE NOTES file in the zipfile for more information. Further information and the library are located in the Google Code Group.

Beginning Version 1.81beta, the library contains MEGA support, although there are some gotchas. Read the release notes and wiki for more info.

The PinChangeInt library implements Pin Change interrupts for the Arduino environment. It is a drop in replacement for the PCint library. This library was designed for the Arduino Uno/Duemilanove, and has been reported to work fine on the Nano, but it has not been tested there. As mentioned above, MEGA support is included but support for that platform is weak.

What are Pin Change interrupts? The ATmega processor at the heart of the Arduino has two different kinds of interrupts: “external”, and “pin change”. There are only two external interrupt pins, INT0 and INT1, and they are mapped to Arduino pins 2 and 3. These interrupts can be set to trigger on RISING or FALLING signal edges, or on low level. The

GreyGnome / PinChangeInt

Code | Issues 6 | Pull requests 0 | Projects 0 | Wiki | Insights

Join GitHub today

GitHub is home to over 28 million developers working together to and review code, manage projects, and build software togethe

Sign up

Pin Change Interrupt library for the Arduino

23 commits | 1 branch | 4 releases | 0 cor

Branch: master | New pull request

Mike Schwager Deprecated the library, and linked to EnableInterrupt.

Examples	Adjusted it so that the sketch is compatible with the EnableInterrupt
LICENSE	Arduino Mega support. Change to Apache2.0 License!
NOTICE	Updates prior to testing for version 2.40-rc1 release. See the
PinChangeInt.h	Defined PCINT_VERSION for version 2.40-rc2.
README	Deprecated the library, and linked to EnableInterrupt.
RELEASE_NOTES	Fix typos.
Technical_Notes	Initial commit. This is my chitchat so I can remember stuff.
keywords.txt	First commit

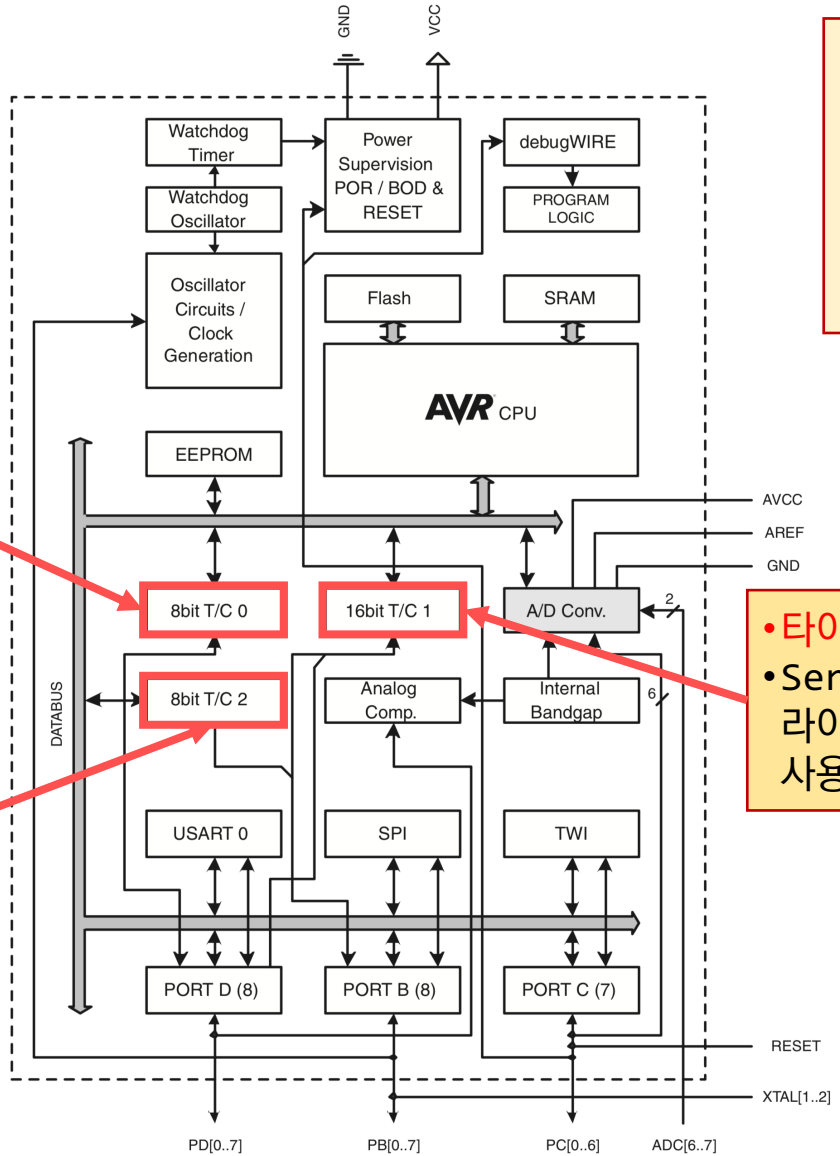
[1] https://playground.arduino.cc/Main/PinChangeInt
[2] https://github.com/GreyGnome/PinChangeInt

타이머 인터럽트

Timer Interrupts

ATmega328P 구조와 타이머 Architecture[1] and Timer

■ 타이머



- Timer0은 8-bit 타이머, 0-255까지 카운트
- Timer1은 16-bit 타이머, 0-65535까지 카운트
- Timer2은 8-bit 타이머, 0-255까지 카운트

- 타이머 0 (Timer0)
- `delay()`, `millis()`, `micros()` 함수에서 사용하는 타이머

- 타이머 2 (Timer2)
- `Tone()` 라이브러리에서 사용하는 타이머

- 타이머 1 (Timer1)
- `Servo()` 라이브러리에서 사용하는 타이머

[1] https://www.microchip.com/wwwproducts/en/atmega328p

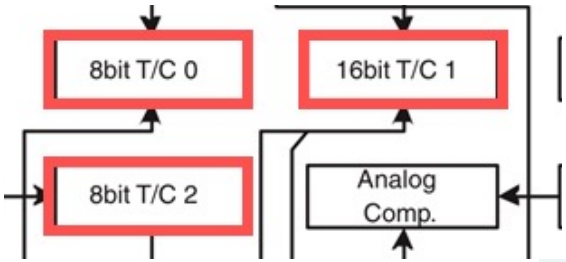
타이머 오버플로우 인터럽트 Timer overflow interrupt (TOI)

■ 타이머 오버플로우와 인터럽트

- 타이머는 입력 펄스를 카운트 하여 최대 값을 (예, 8-bit 타이머일 경우 255) 넘을 경우 0으로 초기화되며 이를 '타이머 오버플로우'라고 함
- 타이머 오버플로우 인터럽트 (TOI)는 타이머가 오버플로우되어 각 비트가 0으로 초기화될 때 마이크로프로세서 내부에서 발생하는 인터럽트
- 각 타이머 (0, 1, 2)의 TOI를 가능하게 하기 위해서는 '타이머 인터럽트 마스크 레지스터 (TIMSKx)'의 TOIE (TOI Enable) 비트를 '1'로 설정
  - ✓ 예로서, 타이머 2의 TOI를 가능하게 하기 위해서는 TIMSK2의 TOIE 비트를 1로 설정

7	6	5	4	3	2	1	0
					OCIEB	OCIEA	TOIE

타이머 인터럽트 마스크 레지스터
Timer's interrupt mask register (TIMSKx)



타이머 오버플로우 인터럽트 Timer overflow interrupt (TOI)

■ 타이머 오버플로우 시간 계산

- 타이머가 오버플로우 하는 시간은 오실레이터[1] 주파수와 클럭 분주 (clock divider 즉, 프리스케일[2])값에 따라 달라짐
- 아두이노 우노는 오실레이터 주파수가 16 MHz 이고 클럭 분주는 64
- 타이머 2의 경우 8-bit 이기 때문에, 오버플로우 시간 $T_{overflow}$는

$$
\begin{aligned}
T_{overflow} &= \frac{1}{F_{osc}} \times clockdiv \times 2^{bits} \\
&= \frac{1}{16000000} \times 64 \times 2^{8} \\
&= 0.001024\ sec
\end{aligned}
$$

- 타이머 2는 TOI는 거의 1 ms 마다 발생
- 즉, 1 kHz 주기로 타이머 오버플로우가 발생

[1] 오실레이터란 일정 주파수를 갖는 클럭을 발생시키는 장치로서 디지털 시스템에서 사용되는 클럭은 오실레이터의 펄스를 분주 (prescale)하여 사용함
[2] 프리스케일이란 마이크로컨트롤러에서 클럭을 그대로 사용하기엔 너무 빠르기 때문에 카운터를 활용해 클럭 속도를 늦추어 (분주하여) 사용하기 위해 설정하는 정수

타이머 오버플로우 인터럽트 Timer overflow interrupt (TOI)

■ 타이머 오버플로우 시간 변경

- 타이머가 오버플로우 하는 시간은 오실레이터[1] 주파수와 클럭 분주 (clock divider 즉, 프리스케일[2])값에 따라 달라짐
- 클럭 분주 값을 바꾸면 TOI 주기를 변경할 수 있음
- '타이머/카운터 제어 레지스터 B (TCCRxB)'를 이용해 클럭 분주 변경
  - ✓ TCCRxB의 2th-to-0th 비트 까지 세 비트를 제어하여 분주 변경

$$T_{overflow} = \frac{1}{F_{osc}} \times clockdiv \times 2^{bits}$$
$$= \frac{1}{16000000} \times 64 \times 2^{8}$$
$$= 0.001024\ sec$$

$$T_{overflow_1024} = 0.001024 \times 16$$
$$= 0.016384 sec$$

CS2 [2:0]	클럭 분주
001 (0x01)	1
010 (0x02)	8
011 (0x03)	32
100 (0x04)	64
101 (0x05)	128
110 (0x06)	256
111 (0x07)	1024

16배

7	6	5	4	3	2	1	0
FOC2A	FOC2B			WGM22	CS2 [2:0]		

타이머/카운터 제어 레지스터 B
Timer/Counter Control Register B (TCCRxB)

TOI 처리 스케치 예제 blink

LED가 빠르게 점등 결과

Practice

```
const int pinLED = 13;
volatile byte state = LOW;

void setup() {
  TIMSK2 = (TIMSK2 & B11111110) | 0x01;

  TCCR2B = (TCCR2B & B11111000) | 0x07;

  pinMode(pinLED, OUTPUT);
}

void loop() {
  digitalWrite(pinLED, state);
}


ISR(TIMER2_OVF_vect){
  state = !state;
}
```

- PCI 인터럽트의 ISR에서 접근할 수 있는 volatile 변수 선언

- 타이머 인터럽트 마스크 레지스터의 경우, 타이머 2를 위한 마스크는 TIMSK2이고, TOI를 가능하게 하기 위해서 TIMSK2의 TOIE 비트를 '1' 설정

- 레지스터 기존 비트들은 그대로 두고 마지막 0-th 비트만 1로 설정

7	6	5	4	3	2	1	0
					OCIEB	OCIEA	TOIE

- 타이머/카운터 제어 레지스터 B (TCCRxB)의 경우, 클럭 분주를 1024로 하기 위해서 CS2 [2:0] 비트를 111 (0x07)로 설정

- 레지스터 기존 비트들은 그대로 두고 마지막 [2:0] 비트만 1로 설정

7	6	5	4	3	2	1	0
FOC2A	FOC2B			WGM22	CS2 [2:0]		

- ISR() 는 아두이노 인터럽트 벡터에 대응하는 ISR 함수
- 인수로는 인터럽트 벡터 값이 올 수 있으며 우노의 경우 ATmega328p 마이크로프로세서를 위해 〈iom328p.h〉 헤더파일에 #define 문으로 정의되어 있음
- 이 예제에서는 타이머 2의 TOI에 해당하는 인터럽트 벡터인 'TIMER2_OVF_vect'를 인수로 전달

[1] https://www.teachmemicro.com/arduino-timer-interrupt-tutorial/

TimerOne 라이브러리

TimerOne 라이브러리 GitHub[1]

PaulStoffregen / TimerOne

Watch 41 | Star 205 | Fork 112

Code | Issues 9 | Pull requests 4 | Projects 0 | Wiki | Insights

Join GitHub today

GitHub is home to over 28 million developers working together to host and review code, manage projects, and build software together.

Sign up

Dismiss

TimerOne Library with optimization and expanded hardware support http://www.pjrc.com/teensy/td_libs_Ti...

32 commits | 1 branch | 1 release | 9 contributors

Branch: master | New pull request | Find file | Clone or download

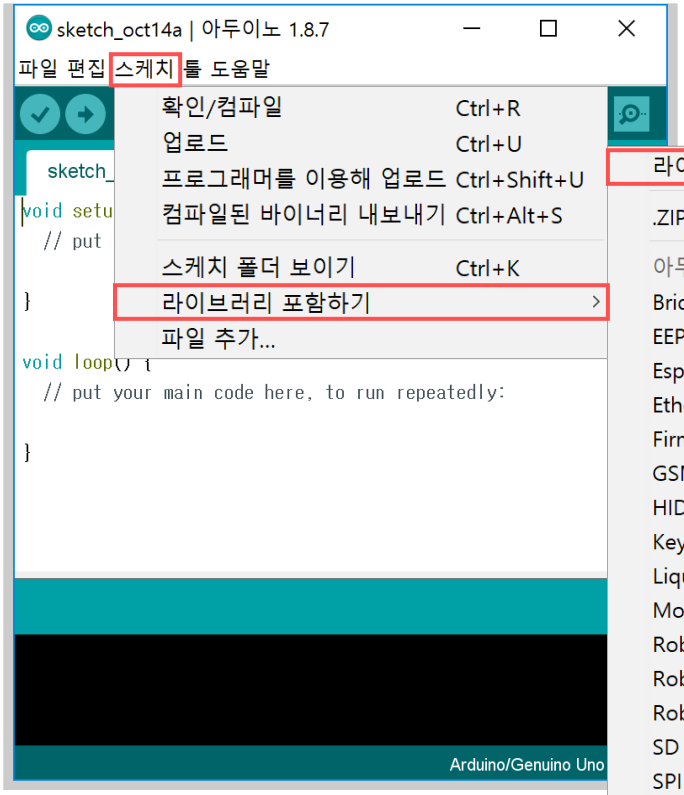
PaulStoffregen Merge pull request #34 from tr4nt0r/master ··· Latest commit 962e1a3 on Sep 27

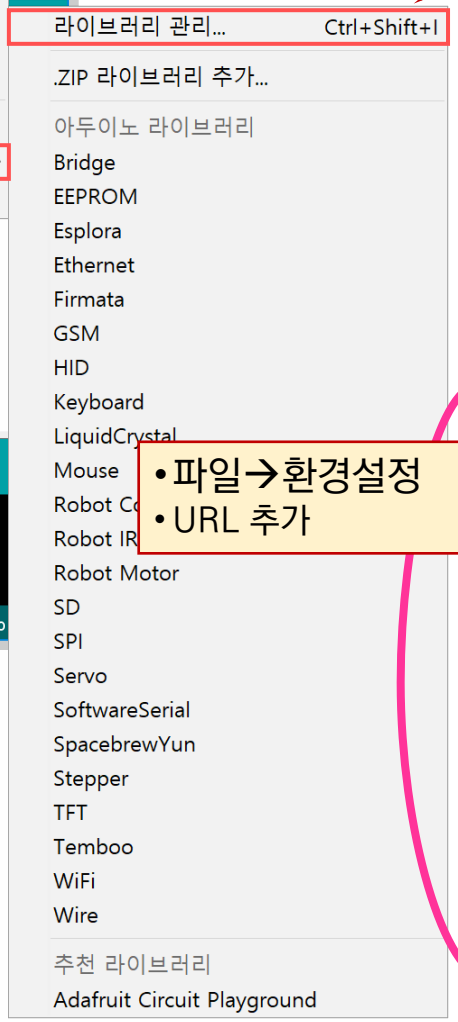
config	Added support for ATmega8	2 months ago
docs	Add issue template	8 months ago
examples	Added support for ATTiny85 except for the PWM functionality	a year ago
README.md	Added support for ATTiny85 except for the PWM functionality	a year ago
TimerOne.cpp	Added support for ATTiny85 except for the PWM functionality	a year ago
TimerOne.h	Added support for ATmega8	2 months ago
keywords.txt	First commit, existing code	5 years ago
library.json	Added support for ATTiny85 except for the PWM functionality	a year ago
library.properties	Added support for ATTiny85 except for the PWM functionality	a year ago

[1] https://github.com/PaulStoffregen/TimerOne/blob/master/TimerOne.cpp

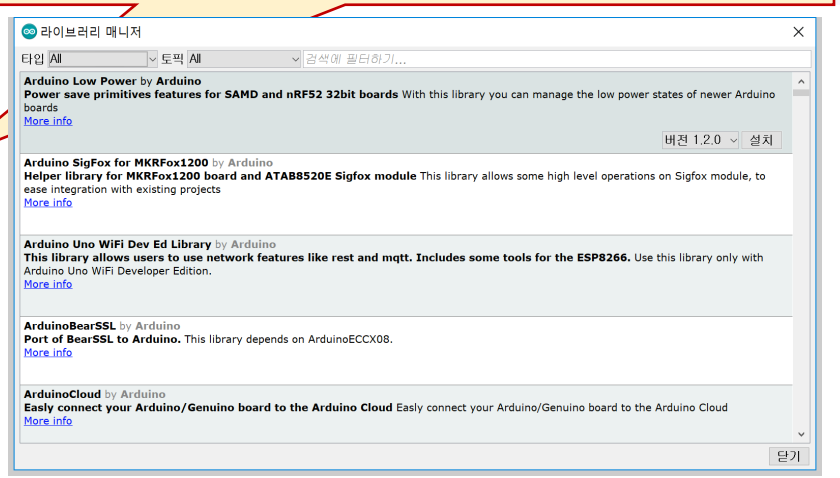
Review: 아두이노 라이브러리 라이브러리 관리

- '보드 매너저 URLs'를 추가하여 라이브러리 관리를 할 수 있음
- 3rd 파티 보드 라이브러리 추가 시 자주 사용

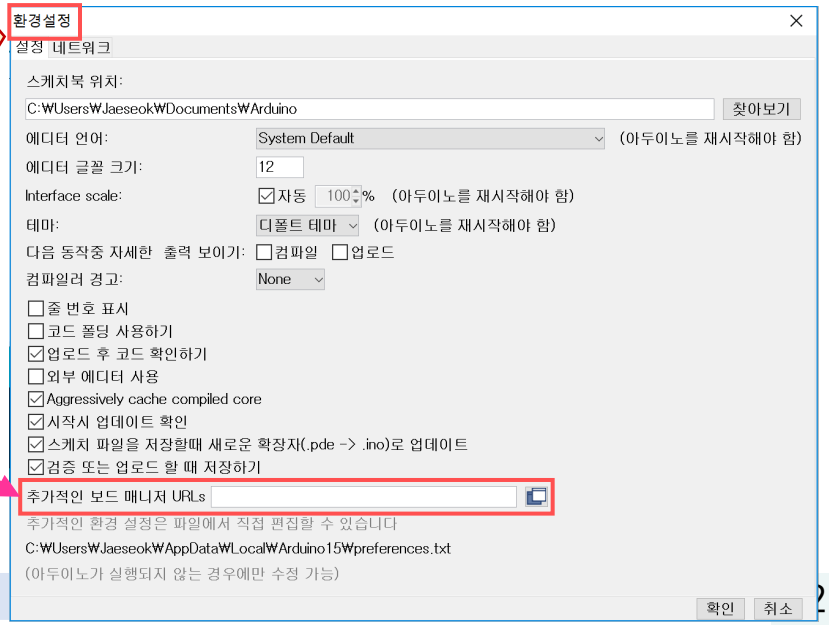




- 파일→환경설정
- URL 추가



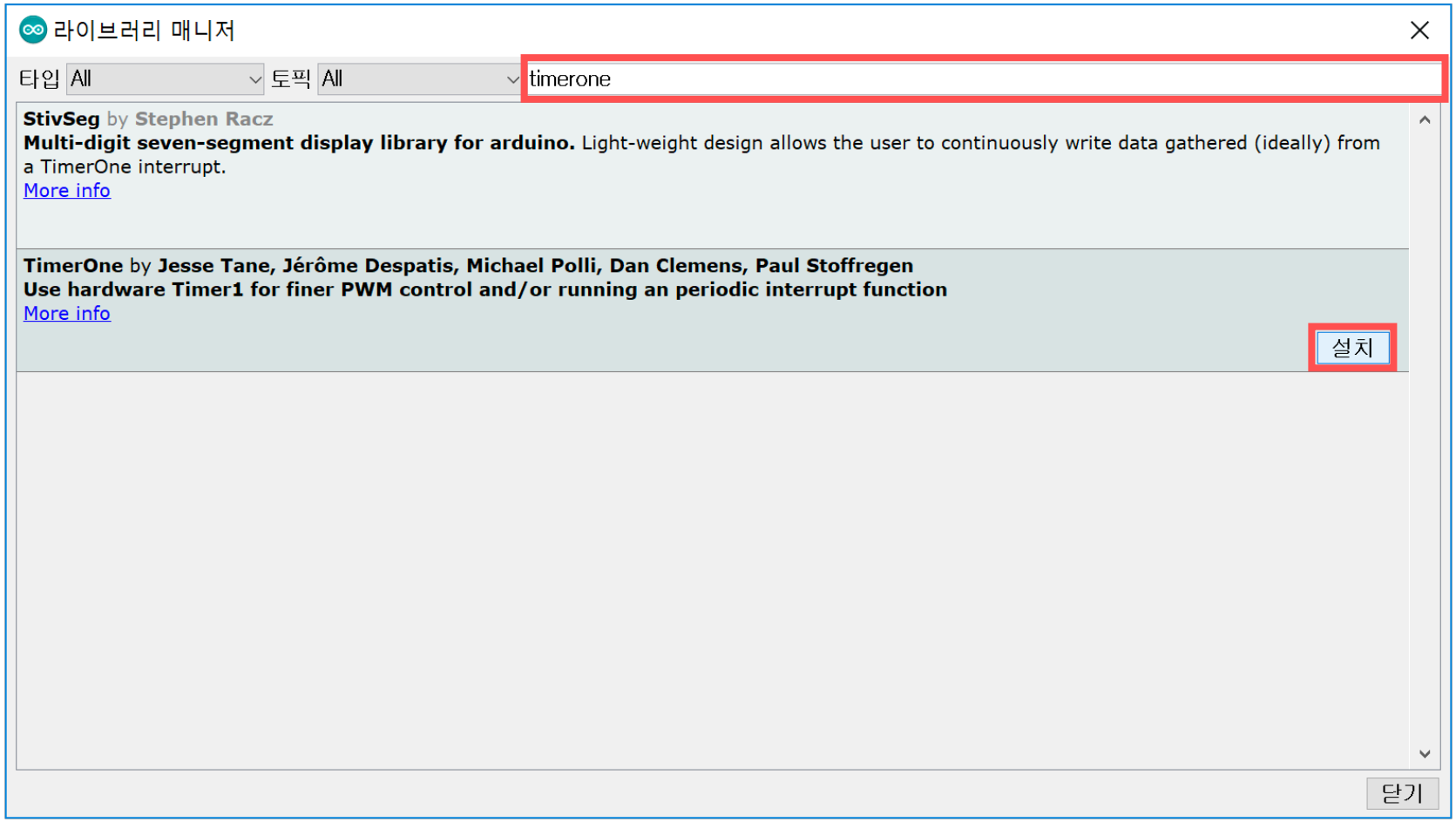
- 예 'Adafruit 라이브러리 URL' 추가할 때 https://adafruit.github.io/arduino-board-index/package_adafruit_index.json



[1] https://www.arduino.cc/en/Guide/Libraries

TimerOne 라이브러리 라이브러리 매니저

- 스케치 → 라이브러리 포함하기 → 라이브러리 관리



TimerOne 스케치 예제

Practice

```
#include <TimerOne.h>

const int pinLED = 13;
volatile byte state = LOW;

void setup() {
  Timer1.initialize(1000000);



  Timer1.attachInterrupt(timerISR);

  pinMode(pinLED, OUTPUT);
}


void loop() {
  digitalWrite(pinLED, state);
}


void timerISR() {
  state = !state;
}
```

- TimeOne 라이브러리 사용을 위해 TimerOne.h 헤더파일을 포함

- initialize() 는 Timer1 인터럽트 주기를 설정
- 인수는 μsec 단위의 unsigned long 숫자 이며, 예로서 1000000이 주어지면, 1초 주기로 인터럽트가 발생

- attachInterrupt() 는 Timer1 인터럽트가 발생했을 때 대응하는 인터럽트 서비스 루틴 (ISR)을 등록
- 인수는 ISR의 이름, 이 예에서는 timerISR
- 결과적으로 매 1초마다 timerISR() 을 호출하는 효과

결과: LED가 1초 주기로 점등

- timerISR() 는 Timer1 인터럽트에 대응하는 ISR 함수
- TimerOne.h 헤더파일에 사용자가 정의한 ISR (여기서는 timerISR()))을 실제 인터럽트 벡터에 대응하는 ISR()로 래핑해주는 부분이 정의 되어 있음[1]

[1] https://github.com/PaulStoffregen/TimerOne/blob/master/TimerOne.cpp

인터럽트 발생 제어

`interrupts()`, `noInterrupts()`

TimerOne 스케치 예제

Practice

```
#include <TimerOne.h>

void setup() {
  Serial.begin(115200);

  Timer1.initialize(1000);
  Timer1.attachInterrupt(timerISR);
}

void loop() {
  noInterrupts();

  Serial.print("(");
  Serial.println(")");

  interrupts();

  delay(10);
}

void timerISR() {
  Serial.println(" ]");
}
```

- TimeOne 라이브러리 사용을 위해 TimerOne.h 헤더파일을 포함
- 시리얼 통신 속도를 올려서 115200 보드레이트 설정
- Timer1 인터럽트 주기를 1000 μs =1 ms로 설정
- timerISR() 함수를 Timer1 인터럽트 ISR로 등록
- timerISR() 는 Timer1 인터럽트에 대응하는 ISR 함수
- loop() 내에서 "(", ")\r\n" 을 시리얼 통신으로 보내는 중간에 인터럽트가 발생하는 경우 "]\r\n" 중간에 끼어들어서 출력
- timerISR() 는 Timer1 인터럽트에 대응하는 ISR 함수

결과

TimerOne 스케치 예제

Practice

```
#include <TimerOne.h>

void setup() {
  Serial.begin(115200);

  Timer1.initialize(1000);
  Timer1.attachInterrupt(timerISR);
}

void loop() {
  noInterrupts();

  Serial.print("(");
  Serial.println(")");

  interrupts();

  delay(10);
}

void timerISR() {
  Serial.println(" ]");
}
```

- TimeOne 라이브러리 사용을 위해 TimerOne.h 헤더파일을 포함
- 시리얼 통신 속도를 올려서 115200 보드레이트 설정
- Timer1 인터럽트 주기를 1000 μs =1 ms로 설정
- timerISR() 함수를 Timer1 인터럽트 ISR로 등록
- **noInterrupts()** 는 인터럽트 (Timer1 인터럽트 포함)가 발생하는 것을 잠시 멈춤
- **interrupts()** 는 인터럽트 발생을 재개
- timerISR() 는 Timer1 인터럽트에 대응하는 ISR 함수
- loop() 내에서 "(", ")\r\n" 을 시리얼 통신으로 보내는 중간에는 인터럽트가 발생하지 않기 때문에 "]\r\n" 가 끼어들지 않음

결과

Summary

- Interrupts and ISR (interrupt service routines)
- External interrupts
- Pin change interrupts (PICINTs)
- Timer overflow interrupts (TOIs)
- TimerOne library
- `interrupts()` and `noInterrupts()`

Q&A

Thank you

Questions?

Contact: eclass.sch.ac.kr
(순천향대학교 학습플랫폼 LMS)